

## EXPERIMENT 2:

### Aim

To write a Python program that solves the 8-Queens problem using the Backtracking algorithm.

### Objective

To represent the 8x8 chessboard and simulate non-attacking queen placements.

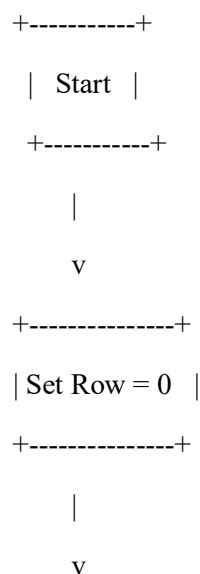
To implement a backtracking algorithm that places 8 queens such that no two queens attack each other (same row, column, or diagonal).

To print the valid board configuration displaying the final queen positions.

### Algorithm

- Initialize an empty 8x8 board (or a 1D array where  $\text{board}[\text{row}] = \text{col}$ ).
- Start at the first row ( $\text{row} = 0$ ).
- For the current row, iterate through each column from 0 to 7:
- Check if placing a queen at  $(\text{row}, \text{col})$  is safe by verifying no other queen shares the same column, main diagonal, or anti-diagonal.
- If safe, place the queen and recursively move to the next row ( $\text{row} + 1$ ).
- If placing a queen in the next row leads to no solution, backtrack by removing the queen from  $(\text{row}, \text{col})$  and try the next column.
- If all 8 queens are successfully placed ( $\text{row} == 8$ ), print the solution board and return true.
- If all columns in the current row are exhausted without a safe position, return false to trigger backtracking.

### Flowchart



```

+-----+
+-----> | Is Row == 8? | ----( Yes )----> +-----+ ----> +-----+
|          +-----+          | Display Board |    | End |
|          |          +-----+    +-----+
|          ( No )
|          |
|          v
|          +-----+
|          | Col = 0 to 7 |
|          +-----+
|          |
|          v
|          +-----+
|          | Is Placement |
|          |   Safe?   |
|          +-----+
|          /      \
|          ( Yes )   ( No )
|          /      \
|          v        v
|          +-----+ +-----+
|          | Place Queen | | Try Next Column / |
|          | at (Row,Col) | |   Backtrack   |
|          +-----+ +-----+
|          |          |
|          v          |

```

## Code

```
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve_nqueens(board, row):
    if row == 8:
        return True
    for col in range(8):
        if is_safe(board, row, col):
            board[row] = col
            if solve_nqueens(board, row + 1):
                return True
    board[row] = -1
    return False

def print_board(board):
    for row in range(8):
        print(" ".join("Q" if board[row] == col else "." for col in range(8)))
    print()

if name == "main":
    board = [-1] * 8
    print("Finding a solution for the 8-Queens problem...\n")
    if solve_nqueens(board, 0):
        print("Solution Board:")
        print_board(board)
    else:
        print("No solution exists.")
```

## Output

Finding a solution for the 8-Queens problem...

Solution Board:

```
Q . . . . . . .  
. . . . Q . . .  
. . . . . Q  
. . . . Q . .  
. . Q . . . .  
. . . . . Q .  
. Q . . . . .  
. . . Q . . .
```

## Result

The 8-Queens program was executed successfully. Using the Backtracking algorithm, it placed 8 queens on an 8x8 chessboard such that no two queens attack each other.

## Conclusion

The Backtracking algorithm efficiently solves the 8-Queens problem. By systematically exploring board configurations and abandoning invalid placement paths early, it dramatically reduces the search space compared to a full brute-force search.